

slice-ephys-mapping — User Guide

How to run the slice PPSF and random-ensemble E/I mapping experiments. Version 1 (Phase A, 2026-08-16) — written against the code as built; the rig-bringup sections will gain measured values in Phase B.

1. Setup

What this software is

Two experiments on patched cells in slices, driven by the DMD photostim path:

- **PPSF** (`exp_slice_ppsf`): patch a cell, measure its direct optogenetic activation (subthreshold Vm + spikes, current clamp) as the stimulation disk varies in **laser power × offset from the soma × DMD fill factor**.
- **Ensemble E/I mapping** (`exp_ensemble_ei`): stimulate thousands of seeded **random ensembles** of opsin+ somata while recording the patched cell; the **identical ensemble set replays** across clamp blocks (current clamp; voltage clamp at −70 mV for excitation and +10 mV for inhibition) so evoked E and I pair per ensemble. Offline Python regression turns the trials into **per-cell weights onto the patched cell** and E/I coupling statistics.

Control is MATLAB (package `sem`, reusing the DMD repo's `tfp` as a library); analysis is Python.

Install / first run

1. Clone this repo **and** `DMD-control-flow-software` as siblings (default expected location `~/code/`). The DMD repo must be a **full checkout** (its `docs/optics_handoff.md` feeds the pattern-safety cap at runtime). If it lives elsewhere: `cp configs/dmd_repo_path_local.m.example configs/dmd_repo_path_local.m` and edit the path.
2. MATLAB R2023a+ (`sem_setup` adds both repos' `src/` to the path — every script below calls it for you).
3. Python: `cd analysis && python3 -m venv .venv && .venv/bin/pip install -e "[dev]"`.
4. Sanity check both suites:

```
bash /Applications/MATLAB_R2023a.app/bin/matlab -nodisplay -batch "runtests" cd analysis && .venv/bin/python -m pytest
```

Mock vs rig configs

Everything runs against a config YAML:

- `configs/mock.yaml` — `hardwareKind: mock`; simulated DMD + a simulated patched cell with **known** connectivity (`groundTruth:` section). Use for development, training, and pipeline validation. No hardware needed.

- `configs/slice_rig.yaml` — `hardwareKind: real`; every hardware line is marked `%VERIFY` until Phase B closes it (section 2).

Configs are flat YAML parsed by a minimal parser: one nesting level only, `key: value` and `[a, b, c]` arrays; no nested maps.

The five-minute smoke test (no hardware)

```
matlab -nodisplay -batch "run(fullfile('scripts','run_mock_session.m'))"
cd analysis
.venv/bin/python -m pytest tests/test_mock_roundtrip.py -v
.venv/bin/python -m slice_ephys_analysis.figures
```

This simulates a full 600-ensemble E/I session from known weights, then proves the analysis recovers them ($r \geq 0.9$ for E, ≥ 0.8 for I) and renders `figures/roundtrip/pdf/mock_roundtrip_validation.pdf`. If this passes, the entire pipeline — patterns → DAQ → trials → files → loaders → regression — is healthy.

2. Rig bringup checklist (Phase B)

Work through `docs/WIRING.md` top to bottom; nothing below it is trusted until its `%VERIFY` marker is removed (the test suite checks the markers still exist until you do this deliberately).

1. **Wire + verify the NI channels** — `run(fullfile('scripts','verify_wiring.m'))` on the rig PC walks: AO identity (laser vs cell-command), the **AO idle-at-last-sample check** (the software-holding design depends on it — if the line does not hold its last sample, STOP), scaled-output gain with a model cell, and the `do_sync` TTL. Update `configs/slice_rig.yaml`
 2. `docs/WIRING.md` as each wire is confirmed.
 3. **Amplifier**: Multiclamp 700B. EXT COMMAND scaling is fixed (VC 20 mV/V, IC 400 pA/V). Read the front-panel/telegraph output gains into `ephys.scaledVcPaPerV` / `scaledIcMvPerMv`. **Commander holding must be 0 in every block** — holding is commanded by the software (section 5).
 4. **Power calibration** — power meter under the objective, then `run(fullfile('scripts','run_power_calibration.m'))`; point `laser.calibration_file` at the saved curve.
 5. **DMD → scan-field affine** — follow the DMD repo's two-step calibration on this rig (`scripts/run_dmd_scan_calibration.m` prints the pointer); set `calibration_file` to the saved calibration. This is what maps segmented 2p centroids onto DMD mirrors.
 6. **Chunk timing** — time a 250-pattern `loadPatternSequence` upload and tune `dmd.chunkSize` (chunk reloads are the only pauses in a block).
 7. **Model cell, then a real patch** — run a mock-config block against the real DAQ; confirm seal-test `Rs/Ri` read correctly before any biology.
-

3. Session workflow: targeting (before the experiments)

Cells come from 2p imaging, **no camera in the loop**:

1. In ScanImage, acquire a reference stack of the opsin channel (opsin is fused to a red fluorophore, soma-targeted).
 2. Segment somata **offline** in your segmentation software (Cellpose / Suite2p / manual). Export centroids in **scan-field coordinates** (the units used during the affine calibration) as a `.csv` (x,y columns) or `.mat` (`centroids`, N×2).
 3. Edit the parameters block of `scripts/run_targeting.m` (segmentation file, optional mean image for click-curation) and run it. It maps centroids → DMD pixels through the calibrated affine, drops cells whose stimulation disk would fall off the chip, assigns cortical layers along the long FOV axis, asks which cell you patched, and writes `targets.mat`.
 4. Pass that path to the experiments as `options.targetsPath`. You can set or change the patched cell later via `options.patchedCellId`.
-

4. Running `exp_slice_ppsf`

Patch a cell (current clamp), then:

```
run('sem_setup.m');
result = sem.experiments.exp_slice_ppsf('configs/slice_rig.yaml', ...
    'ppsf_cell1', struct('targetsPath', 'data/targeting/targets.mat', ...
        'patchedCellId', 12, 'interactive', true));
```

- The factorial comes from the `ppsf` config section: `laserVoltsLevels × fillFactors × a Gaussian-spaced offset grid ((2·nPointsPerHalfAxis+1)2 points, dense near the soma, out to offsetsMaxUm) × nReps`, seeded shuffle.
- One CC block; a seal test opens and closes it and repeats every `ephys.sealTest_everyNTrials` trials.
- `result.trialTable` is the quick look: per trial `distanceUm`, `laserVolts`, `fillFactor`, `peakDepolMv`, `nSpikes`. The rigorous PPSF surfaces/half-widths come from Python (`slice_ephys_analysis.ppsf`).
- **When to abort:** Rs warnings in the console (limit `ephys.rsAbortMohm`, drift `rsDriftAbortFrac`), a drifting patch, or anything odd on the monitor scope. `Ctrl-C` is safe: the DAQ session closes via cleanup, and every completed trial is already on disk at block finalize.

5. Running `exp_ensemble_ei`

```
run('sem_setup.m');
result = sem.experiments.exp_ensemble_ei('configs/slice_rig.yaml', ...
    'ei_cell1', struct('targetsPath', 'data/targeting/targets.mat', ...
        'patchedCellId', 12, 'seed', 20260901, ...
        'interactive', true));
```

What happens, in order:

1. **Ensemble generation** (`ensemble:` config section): `nEnsembles` random ensembles (size `sizeMin..sizeMax`) of opsin+ cells (never the patched cell), each member at a random fill level from `fillLevels` — fill factor IS the per-cell power knob (one fixed `laserVolts` per session). Plus `nBlank` catch trials. Everything is written to `ensembles.mat` **before** any stimulation — that file is the design matrix the analysis trusts.
2. **Blocks** (`ensemble.blocks` , e.g. `[CC, VC-70, VC+10]`): the SAME ensembles in every block, order reshuffled per block. Between blocks the software pauses and prompts you to switch the **Commander mode** (VC/IC); the **holding potential is commanded automatically** through the cell-command AO (ramped over ~100 ms; Commander holding stays 0).
3. **Within a block** (~6 Hz at default timing: 10 ms pulses, `itiMeanS ± itiJitterS`): patterns preload to the DMD in chunks of `dmd.chunkSize`; each trial advances the pattern and queues one hardware-clocked AO waveform (laser pulse + holding). Seal tests interleave every `sealTest_everyNTrials` . A failed trial is recorded and the block continues; hardware errors abort the block.
4. **Timing expectations**: 3000 ensembles at ~6 Hz ≈ 9 min per block plus ~12 chunk reloads; three blocks with mode switches ≈ 35–45 min per cell. Watch the Rs warnings between blocks.
5. Cells for the E vs I comparison: record **E cells at -70** and **I at +10** in whichever cells you hold long enough — pass `options.blocks = {'VC-70', 'VC+10'}` (or a single block) to run subsets.

6. Data layout

`data/<sessionName>/` — see `docs/DATA_SCHEMA.md` for every field:

```
config_snapshot.mat/.yaml    targets.mat    ensembles.mat    ground_truth.mat (mock)
log.txt                      block_<k>_<label>/session.mat (uncut record)
block_<k>_<label>/trials/trial_%04d_{meta,row}.mat
```

Key invariants: raw `aiData` is **DAQ volts** (the gains snapshot in `metadata.ephys` converts it, in exactly one place per language); `metadata.ephys.ensembleId` joins trials to `ensembles.mat` rows and pairs E with I across blocks; DMD patterns are never stored — they regenerate exactly from `ensembleSeed` .

7. Analysis (Python)

```
cd analysis
.venv/bin/python
>>> from slice_ephys_analysis import io_mat, design, regression, ei_coupling
>>> idx = io_mat.SessionIndex('data/ei_cell1')
>>> d_e = design.average_repeats(design.block_design(idx.block_by_holding(-70), idx.ensembl
>>> d_i = design.average_repeats(design.block_design(idx.block_by_holding(10), idx.ensembl
>>> fit_e = regression.fit_weights(d_e['X'], d_e['y'], n_permutations=100) # W_E per cel
>>> fit_i = regression.fit_weights(d_i['X'], d_i['y'], n_permutations=100) # W_I per cel
>>> paired = ei_coupling.pair_by_ensemble(d_e, d_i)
>>> ei_coupling.coupling_stats(paired) # r, I~E fit, outlier ensembles
```

- Weights are pC of evoked charge per unit fill fraction; `null_hi` is the permutation band — cells inside it are "not detected".
- `blank_sigma` in each design is the catch-trial noise floor.
- PPSF sessions: `slice_ephys_analysis.ppsf.ppsf_table` / `lateral_profile` / `half_width_um`.
- Figures (`slice_ephys_analysis.figures`) follow the lab standards automatically: PDF + PNG + clickable source-code listing, overlap-checked.
- **Trust but verify:** after ANY pipeline change, rerun the mock roundtrip (section 1) before applying it to real data.

8. Safety + troubleshooting

- **Pattern safety cap:** every DMD upload is checked against the optics handoff's 50% on-fraction cap (`tfp.hardware.DMD.onFractionExceeded`) — mock included. Never bypass; ensembles are ~4% ON.
- **Abort:** `Ctrl-C`, or `tfp.util.safetyChecks('abort')` from a second MATLAB prompt — checked before every trial.
- **Holding is software-owned:** if the amplifier behaves oddly between trials, first confirm Commander holding = 0 and re-run `verify_wiring.m`'s AO-idle check.

Symptom	Likely cause / fix
<code>sem:setup:dmdRepoNotFound</code> / <code>handoffMissing</code>	DMD repo missing or partial checkout — fix <code>configs/dmd_repo_path_local.m</code>
<code>tfp:io:loadConfig:badYaml</code>	nested map beyond one level, or missing <code>hardwareKind</code>
<code>tfp:hardware:DMD:onFractionExceeded</code>	a pattern above the 50% cap — check radius/fill/centroid inputs
Rs warnings (<code>rsAboveLimit</code> / <code>rsDrift</code>)	patch degrading — decide re-patch vs discard block
<code>sem:targeting:buildTargets:cellsOutsideDmd</code>	affine wrong or FOV off-chip — redo the two-step calibration
Roundtrip test failing after a code change	you changed the physics/analysis contract — fix before real data
Python loader returns <code>None</code> fields	a MATLAB <code>string</code> / <code>datetime</code> leaked into the schema — use <code>char</code> /double

Regenerate the PDF after editing this file: `scripts/render_user_guide.sh`.